4-3 · MinIO–Iceberg 데이터 파이프라인 구축 · 정리·회고 0.5H

정리·회고

파이프라인 다섯 단계를 한 장으로, 4-4 준비

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

① 오늘 만든 파이프라인을 랜딩·읽기·형 변환·정제·적재 다섯 단계로 말한다

② 4-4 Trino 가 읽을 orders 테이블의 행 수를 적어 둔다

③ 자주 겪는 오류 다섯 가지의 원인과 해결을 안다

④ 4-4 Trino 준비물(8080 포트 · 이미지 pull)을 확인한다

2-5 의 fetch·parse·load 와 같은 세 덩어리 — 규모와 도구가 달라져도 구조는 유지됩니다

오늘의 요약

오늘 만든 파이프라인 — 다섯 단계

단계	무엇을	어디에
① 랜딩	CSV 를 받은 그대로	s3://raw/orders/
② 읽기	s3a 로 접두사 통째로	Spark DataFrame
③ 형 변환	cast · to_date	메모리
④ 정제	불량 제거 + 키별 최신 한 행	메모리
⑤ 적재	MERGE INTO (있으면 갱신·없으면 삽입)	Iceberg 테이블

읽기(②) · 정제(③④) · 적재(⑤) — 2-5 의 fetch·parse·load 와 같은 세 덩어리

다음 차시를 위한 상태 확인

4-4 Trino 가 읽을 테이블 — 지금 행 수를 적어 둔다

1. **spark-sql -e "SELECT count(*) FROM demo.lakehouse.orders;"**
   6 이면 된다 — order_id 1~5, 7

2. **spark-sql -e "SELECT * ... ORDER BY order_id;"**
   4 choi 61000.0 app · 7 ok 18000.0 app · 6 은 걸러져 없다

3. **docker compose exec mc mc ls local/raw/orders/**
   orders_bad.csv 는 빠져 있고 08-01 · 08-02 · 08-03 세 파일

적어 둘 숫자

orders

6행

오늘의 체크리스트

여덟 개를 스스로 확인해 보세요

1. 랜딩 존을 따로 두는 이유를 두 가지 이상 말할 수 있다
2. 멱등성이 왜 파이프라인의 기본인지 설명할 수 있다
3. s3a 에 필요한 --packages 하나와 --conf 네 개를 설명할 수 있다
4. ETL 을 짜기 전에 spark-shell 로 읽기부터 확인했다
5. 중복을 안 줄이고 MERGE 해서 카디널리티 오류를 직접 봤다
6. dropDuplicates 대신 ROW_NUMBER 로 결정적 중복 제거를 구현했다
7. 같은 스크립트를 두 번 실행해도 행 수가 늘지 않음을 확인했다
8. 새 파일만 추가한 증분 배치에서 불량 행이 걸러짐을 확인했다

자주 겪는 오류 ①

s3a 가 시작부터 막힐 때

증상	원인	해결
ClassNotFoundException S3AFileSystem	이 이미지에 hadoop-aws (s3a 부품)가 없다	--packages 줄 추가 hadoop-aws:3.3.4
시작 직후 몇 분간 출력 없이 멈춤	첫 실행이라 부품 275MB 를 받는 중	기다리기 · 미리 받기 down 은 지움 · stop 은 유지
오류 없이 6분 넘게 멈춤	path.style.access 빠짐 raw.minio 를 찾는 중	Ctrl+C 뒤 --conf 네 줄 확인

멈춤은 오류 문구가 없다 — 로그에 downloading 줄이 보이면 부품을 받는 중

자주 겪는 오류 ②

읽기·적재가 실패할 때

증상	원인	해결
s3a 읽기 403 AccessDenied	access.key·secret.key 누락 또는 오타	--conf 4종 재확인 http://minio:9000
[MERGE_CARDINALITY_ VIOLATION]	원천에 같은 order_id 여러 행	MERGE 전 ROW_NUMBER 로 키별 한 행만
실행마다 결과가 미묘하게 다름	순서 없는 중복 제거 (dropDuplicates)	Window + row_number 로 규칙 명시

dropDuplicates 는 어느 행이 남을지 보장하지 않는다 — 규칙을 명시해야 실행마다 같은 결과

예상 질문

자주 나오는 질문 네 가지

1. **매번 랜딩 존 전체를 읽으면 느리지 않나요?**
   느려진다 — 날짜 파티션 경로만 읽거나 처리한 파일을 다른 접두사로 옮긴다. 8주차 Airflow 에서

2. **MERGE 대신 INSERT OVERWRITE 는요?**
   파티션 단위로 통째로 갈아 끼우는 방식이라 그것도 멱등 — 갱신 대상이 흩어져 있으면 MERGE 가 낫다

3. **원천에 삭제가 오면요?**
   WHEN MATCHED AND s.op = 'D' THEN DELETE 를 붙인다 — 6주차 CDC 에서

4. **불량 행을 그냥 버려도 되나요?**
   권하지 않는다 — 별도 테이블에 남겨 두면 「왜 건수가 안 맞나」를 나중에 설명할 수 있다

심화 과제 (선택)

더 해 보고 싶다면

1. **격리 테이블**
   걸러진 불량 행을 demo.lakehouse.orders_rejected 에 따로 적재 — 3-5 품질 리포트와 같은 발상

2. **컴팩션**
   CALL demo.system.rewrite_data_files 로 작은 파일을 합치고 files 메타테이블로 전후 비교

3. **날짜 경로**
   랜딩 존을 dt=날짜 형태로 바꾸고 특정 날짜만 읽는 인자 추가

4. **「최신」의 기준 바꾸기**
   order_date 대신 파일 이름으로 바꾸면 결과가 어떻게 달라지는지

다음 회차 예고 · 10-21(수)

4-4 · Trino 기반 S3 SQL 엔진 구축

01

이론 1H

분산 SQL 엔진의 이해

코디네이터·워커

02

실습 1.5H

Trino 컨테이너와

카탈로그 연결

03

실습 1H

Trino SQL 실습

집계·CTAS·타임 트래블

04

정리 0.5H

오늘 배운 것을

한 장으로

준비: 4개 컨테이너 up · orders 6행 확인 · 8080 포트 비우기 · docker pull trinodb/trino 미리

4-3 · 정리·회고 0.5H 마무리

다음 회차는 4-4 Trino SQL 엔진

오늘 Spark 로 적재한 그 테이블을, Trino 가 아무것도 옮기지 않고 그대로 읽는 것을 확인합니다

- ✓ 코디네이터·워커 구조와 커넥터
- ✓ 저장과 계산이 분리된 구조의 장점
- ✓ iceberg.properties 로 카탈로그·MinIO 연결
- ✓ Trino CLI 로 집계·CTAS·타임 트래블

준비할 것

orders 6행 · 8080 포트 · trinodb/trino pull